

An Empirical Validation of Object-Oriented Metrics in Two Different Iterative Software Processes

Mohammad Alshayeb, *Member, IEEE*, and Wei Li, *Member, IEEE*

Abstract—Object-oriented (OO) metrics are used mainly to predict software engineering activities/efforts such as maintenance effort, error proneness, and error rate. There have been discussions about the effectiveness of metrics in different contexts. In this paper, we present an empirical study of OO metrics in two iterative processes: the short-cycled agile process and the long-cycled framework evolution process. We find that OO metrics are effective in predicting design efforts and source lines of code added, changed, and deleted in the short-cycled agile process and ineffective in predicting the same aspects in the long-cycled framework process. This leads us to believe that OO metrics' predictive capability is limited to the design and implementation changes during the development iterations, not the long-term evolution of an established system in different releases.

Index Terms—Software metrics, agile process, empirical validation, framework evolution.

1 INTRODUCTION

MANY studies evaluated quantitative software product metrics in the object-oriented paradigm [2], [8], [10], [18]. Other studies tested the validity of OO metrics in indicating project progress and evolutionary changes; such studies focused on using OO metrics in monitoring the evolution and progress of software projects and controlling them [26], [27], [28].

Metrics were used in conjunction with prediction models to predict different aspects of software such as effort. Basili et al. proposed an approach that was successfully used to predict an effort model for software maintenance releases [3]. Nesi and Querci presented new complexity and size metrics that can be used for effort evaluation and prediction [9]. Moser et al. reported results from an empirical study and proposed a new approach named System Meter (SM) to predict effort [11]. Hastings and Sajeew introduced a Vector Size Measure (VSM) that incorporates both functionality and problem complexity and measures the size of software systems, classifies software systems, and estimates development effort early in the software life cycle [12]. Briand and Wüst investigated the relationship between class size and development effort, proposed a practical and accurate analysis procedure to investigate relationships between structural properties and development effort, and concluded that fairly accurate predictions of class effort can be made based on simple measures of the class interface size alone [13].

Researchers have also tried to predict maintenance. Rombach concluded that most of the maintenance aspects can be explained and predicted sufficiently well by the use of metrics [14], [16]. Li and Henry identified metrics that can predict maintainability [18], [6]. Fioravanto and Nesi employed a set of several well-known metrics and proposed a model for the estimation/prediction of adaptive maintenance effort [7].

Other researchers questioned the predictive capabilities of metrics and prompted questions about their use. Fenton and Neil identified a number of theoretical and practical problems in a critical review of the literature on software metrics and the associated statistical models [20]. Fenton and Ohlsson did not find any evidence that the popular complexity metrics are good predictors of either fault-prone or failure-prone modules [22].

These studies yielded mixed results with some studies confirming the predictive capabilities of the metrics and others prompting questions about these metrics. Furthermore, none of the studies investigated the effectiveness of OO metrics in iterative processes for continuously evolving systems. Using two sets of data collected from a commercial environment and several releases of a framework system, we conducted an empirical validation of the metrics to examine their predictive capabilities in two iterative processes: the short-cycled agile process and the long-cycled framework evolution process.

Agile software processes suit small to medium-sized teams better than traditional processes and are more flexible as well [1]. Beck introduced Extreme Programming (XP) as a new agile process [4]. XP is an iterative software process and it is especially convenient and effective for projects that have vague or changing requirements. The XP development process has various unique features such as stories, pair programming, test-driven design, and frequent unit and integration tests. XP emphasizes short iteration cycles, which are triggered by stories. A story is a set of

• The authors are with the Computer Science Department, University of Alabama in Huntsville, Huntsville, AL 35899.
E-mail: mohammad@alshayeb.com, wli@cs.uah.edu.

Manuscript received 12 Aug. 2002; revised 8 Aug. 2003; accepted 11 Aug. 2003.

Recommended for acceptance by A. Mili.

For information on obtaining reprints of this article, please send e-mail to: tse@computer.org, and reference IEEECS Log Number 117117.

requirements supplied by users/customers. XP is a test-driven iterative process where test case design precedes the architectural and system design in each iteration cycle.

There are three main activities in each XP iteration cycle: *new design*, *error fix*, and *refactoring*. Adding new classes or methods are considered as new design effort. Error-fix corrects deficiencies uncovered during the development. Refactoring effort changes system architecture because programmers believe that the new architecture is better based on their experience.

Framework systems are widely used in commercial software production. Some well-known framework systems are Microsoft Foundation Classes (MFC) in C++, and Java Foundation Classes (JFC) or Java Development Kit (JDK) in Java. We choose all JDK releases as the subject of our study because JDK is widely used in the software industry with several years of history and JDK is an open-source system. With four major releases and the substantial size of JDK, we can verify the effectiveness of the metrics with confidence.

2 RESEARCH OBJECTIVES AND HYPOTHESES

Most, if not all, software systems are developed in iterative processes. Although some metrics studies linked OO metrics to certain system quality factors, the metrics studies so far have not investigated the effectiveness of metrics in iterative processes. It is reasonable to think that if OO metrics are linked with some software quality factors in the cause-effect relationship, OO metrics should be linked with these quality factors in all forms of iterative processes. In this study, we examine the predictive capabilities of OO metrics in two different iterative processes. Our goal is to investigate the relationship between metrics and development/maintenance efforts in two forms of iterative processes, thus validating/invalidating OO metrics. In situations where we do not have maintenance effort data, we use system change data as surrogate because we can reasonably believe that more changes in a system mean more effort; three types of changes: lines added (LA), changed (LC), and deleted (LD), are identified as surrogates of maintenance effort. We set up the following hypotheses to focus our study and to facilitate statistical analyses:

Hypothesis 1: Using OOmatics, we can predict added, changed, and deleted source lines of code in classes from one iteration (release) to the next in the long-cycled framework iterative process.

Hypothesis 2: Using OOmatics, we can predict added, changed, and deleted source lines of code in classes from one iteration to the next in the short-cycled XP iterative process.

Hypothesis 3: Using OO metrics, we can predict maintenance effort (measured in man-hours) in classes from one iteration to the next in the short-cycled XP iterative process.

Hypothesis 4: Using OO metrics, we can predict refactoring effort (measured in man-hours) in classes from one iteration to the next in the short-cycled XP iterative process.

3 EMPIRICAL STUDY

The goal of this study is to investigate the relationship between OO metrics and development/maintenance effort by testing the hypotheses. We use Multiple Linear Regression (MLR) [5] to predict LA, LC, and LD from some selected metrics.

3.1 Multiple Linear Regression

MLR investigates the relationship between a dependent variable and multiple independent variables. The MR equation takes the form:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_k x_k + \varepsilon, \quad (1)$$

where

- The x s are the independent (predictor or regressor) variables.
- y is the dependent (response) variable.
- ε —the statistical error—is the difference between the observed value y and the line $(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_k x_k)$ [15], [21].
- β (regression coefficient) is the average amount of dependent increase when the independent increase one unit one unit and other independents are held constant.

The same assumptions that apply to correlation apply to MLR. Such assumptions include linearity of relationships, the same level of relationship throughout the range of the independent variable, interval or near-interval data, and data whose range is not truncated. The tested model must be correctly specified.

The accuracy of the model can be measured by the coefficient of determination R^2 , which is the percentage of the variance in the dependant explained by the independents.

3.1.1 Regression Terms and Concepts

- Predicted (fitted) values: The values of each case based on using the regression equation for all cases in the analysis.
- The regression coefficient (β): If all independents except a given independent are held constant while the given independent variable is increased by the amount of one unit, the β coefficient would be the average amount that a given dependent increases under those conditions. Another way of defining β is as the regression line slope. That is, the slope gets steeper as β gets bigger and vice versa.
- R^2 (coefficient of determination): The variance percent in the dependent variable based on the knowledge of the independents. R^2 is the proportionate decline in error in estimating the dependent while knowing the independents. This means that R^2 is the number of errors made when guessing the dependent variable value using the regression model, divided by the overall number of errors made when estimating all cases with the dependent's mean as the basis.

$$\begin{aligned} \text{Mathematically: } R^2 &= (1 - (\text{SSE}/\text{SST})), \text{ where} \quad (2) \\ \text{SSE} &= \text{error sum of squares} \\ &= \text{SUM}((Y_i - \text{Est}Y_i)^2); \end{aligned}$$

Y_i is the actual value of Y for the i th case;
 $EstY_i$ is the regression prediction for the i th case;
 SST = total sum of squares
 $= \sum((Y_i - \text{Mean}Y)^2)$.

- Maximizing R^2 : Trying to maximize R^2 by adding variables is generally not suitable unless there are firm theoretical reasons for including the additional variables. As a maximum, with $n-1$ additional variables R^2 is 1, but this result is meaningless. To penalize for adding variables, a conservative reduction to R^2 is used, adjusted R^2 .
- Standard Error of Estimate (SEE), confidence intervals, and prediction intervals: In regression, confidence refers to more than one thing. The confidence and prediction intervals improve (narrow) when the confidence level is decreased (e.g., from 95 to 90 percent), or the sample size is increased.
- The confidence interval of the regression coefficient: Using t -tests as a basis, the confidence interval can be defined as a range (plus/minus) around the observed sample regression coefficient. We can be approximately 90 percent confident that the actual population regression coefficient falls in that range. Confidence limits are only applicable to random samples. If 0 occurs in the confidence interval, then no considerable linear relationship between x and y exists. In that case, the null hypothesis that x is independent of y cannot be rejected.
- t -tests: t -tests are used to measure the significance of individual β coefficients. Particularly, t -tests test the null hypothesis stating that the regression coefficient equals zero. A generally applied rule is dropping from the equation all variables not significant at the .05 level or better. Restricted variance of the independent variable in the specific given sample may lead to a result of no significance. Moreover, similar to other significance tests, the t -test assumes a random data sample.

3.2 Selected Object Oriented Metrics

Because of the huge amount of data, especially in the JDK releases, we consider tool support a critical factor in the research. We use metrics that are published in literatures and can be automatically collected from source code. The selected metrics, which are used as independent variables in MLR, are as follows:

1. Weighted methods per class (WMC): The WMC is the sum of the complexities of all local methods [10].
2. Depth of inheritance tree (DIT): This metric measures how many ancestor classes in the class hierarchy can affect this class. Classes with high values of DIT tend to complex classes [10].
3. Lack of cohesion of methods (LCOM): A measure of the count of pairs of the instance variable set [10].
4. Number of Local Methods (NLM): The number of the local methods, which are accessible outside the class (public methods) [19].

5. Coupling Through Abstract Data Type (CTA): The total number of classes that are used as abstract data types in the data attribute declaration of a class [19].
6. Coupling Through Message Passing (CTM): This measures the number of different messages sent out from a class to other classes, excluding the messages sent to objects created as local objects in the local methods of the class [19].

The Chidamber and Kemerer (C&K) metrics suite is a widely used metrics suite. It has been validated by several researchers [2], [18], [24] and it forms the core of our study. However, researchers identified some deficiencies in some of the C&K metrics. Nevill et al. pointed out potential problems with some of the C&K metrics and suggested alternatives [23]. Hitz and Montazeri argued that it is premature to apply C&K metrics because of unresolved uncertainty about the precise definitions of many quantities to be observed and their impact upon subsequent indirect metrics [17]. Li's suite of OO metrics solved deficiencies in some of the C&K metrics and has alternatives to some of their measures [19].

We selected the C&K metrics because of their wide acceptance among the software engineering community; numerous previous empirical studies used them. Due to deficiencies found in some of the C&K metrics, we also used some metrics from the Li metrics suite, which was proposed to remedy some of the C&K metrics deficiencies.

These metrics are collected with the help of Webgain's Quality Analyzer and Krakatau Project Manager.

3.3 Application of Multiple Linear Regression

Using Minitab software, we applied MLR to calculate the R^2 values of lines added, deleted, and changed and the refactoring and error fix efforts for each story. We provide Minitab with a series of values of independent and dependent variables (x s and y s) for each class in the story. Based on those values, β s and ε are calculated giving a prediction equation in the form of (1), and R^2 is calculated using (2). Dependent variables are added, deleted, and changed lines of code as well as the refactoring and error fix efforts for each class in each story, while independent variables are the WMC, DIT, LCOM, NLM, CTA, and CTM OO metrics for each class in each story.

For example, to obtain the R^2 value of lines added for Story n , we provide:

- Actual lines added for each class between Story $n-1$ and Story n (series of y s).
- The values for the WMC, DIT, LCOM, NLM, CTA, and CTM metrics for each class in Story $n-1$ (series of x s).

Minitab uses those values to generate the prediction equation for lines added for Story n in the form of (1), and the R^2 value of lines added for Story n . We use this R^2 value as an indication of the accuracy of the prediction model.

An example of an actual Minitab generated prediction equation for lines added in System 2 between Story 4 and Story 5 is:

$$LA = 450 - 212 \text{ DIT} - 15.3 \text{ CTM} + 57.0 \text{ NLM} + 2.05 \text{ CTA}.$$

TABLE 1
JDK Releases and Number of Files in Each Release

JDK Version	Added Files	Deleted Files	Total
JDK 1.0	216	0	216
JDK 1.1	462	0	678
JDK 1.2	1088	141	1625
JDK 1.3	223	8	1840
JDK 1.4	1895	19	3716

This equation is generated by providing Minitab with:

- actual lines added for each class between Story 4 and Story 5 in System 2 and
- the values for the WMC, DIT, LCOM, NLM, CTA, and CTM metrics for each class in Story 4 of System 2.

Based on this data, Minitab calculated the R^2 value of lines added for Story 5 in System 2 to be 97.7 percent indicating a high prediction accuracy.

3.4 Research Data

We use three sets of data for this research: one for the framework evolution process and two for an XP-like process. In the first data set, we collected four major JDK releases ranging from JDK 1.0 through JDK 1.4. Table 1 shows JDK releases and the number of files added/deleted in each release.

In the other two data sets, we collected system evolution data from two client-server systems over a 3-year period; data collection from the first system happened over a period of 18 months, while data collection from the second system happened over a period of six months. The two systems were developed primarily using Java, Java Servlets, and XML. The two systems were delivered as

commercial products to customers. Both systems were developed using an agile process similar to XP with several short iteration cycles and customer-supplied stories; each story marked the beginning of a new cycle. The programmers/designers planned and designed for a story at the beginning of each cycle.

Data from the two systems collected from *research logs*. The research logs of the two systems recorded daily activities. A log file was created for each working day during system development. The information recorded in each log file includes:

- The tasks planned for the current iteration cycle. The tasks come from the user stories, which are supplied by the customer.
- A description of the progress or failure of progress towards completing the tasks.
- The time spent on each task measured by hours.
- A description of the problems encountered during implementing a task and the attempted solutions.
- A description of the changes made on the system, the reasons for the changes, and the affected classes.

Table 2 and Table 3 summarize the two data sets, respectively. Table 4 shows an example of inserted, changed, and deleted lines of code as the result of using the PM tool. The strikethrough line indicates a deleted line between two versions of the same file. The line with italic font shows an inserted line. The underlined line is a changed line.

4 RESEARCH RESULT

To test the hypotheses defined in Section 2, we used the MLR statistical technique, which investigates the relationship between a dependent variable and multiple independent variables. By using the set of metrics described in

TABLE 2
System 1—Number of Classes in Each Story

Story Number	Added Classes	Deleted Classes	Changes in Class Names	Total Number of Classes
Story 1	1	0	0	1
Story 2	2	0	0	3
Story 3	5	2	0	6
Story 4	2	0	1	8
Story 5	11	0	1	19
Story 6	14	3	2	30

TABLE 3
System 2—Number of Classes in Each Story

Story Number	Added Classes	Deleted Classes	Changes in Class Names	Total Number of Classes
Story 1	2	0	0	2
Story 2	3	1	1	4
Story 3	5	2	1	7
Story 4	14	4	1	17
Story 5	2	0	2	19
Story 6	34	1	3	52
Story 7	10	12	0	50

TABLE 4
Example of Lines Added, Deleted, and Changed

<pre>// old file public class Example{ public static void main(String args[]){ System.out.println("Welcome to java"); System.out.println("Hello"); System.out.println("World"); } }</pre>
<pre>// new file public class Example{ public static void main(String args[]){ System.out.println("Hello"); System.out.println("Java"); System.out.println("new World"); } }</pre>

TABLE 5
MLR Results for the JDK Systems

Dependent Variables	Acronyms	JDK1.0-1.1	JDK1.1-1.2	JDK1.2-1.3	JDK1.3-1.4
		Sample size =113	Sample size =234	Sample size =483	Sample size =498
		R ² %	R ² %	R ² %	R ² %
Lines Added	LA	42.6	10.6	27.8	32.7
Lines Changed	LC	56.0	18.6	24.9	13.6
Lines Deleted	LD	58.0	11.0	12.9	7.30

Section 3 and the line differences between one release and another, we calculate the R² value. R² calculates the percentage of the variance in the dependant explained by the independents, which shows the statistical significance of the prediction model.

To test Hypothesis 1, we present the result of MLR in Table 5. We noticed that almost none of the dependent variables could be predicted with an acceptable accuracy, thus, rejecting the hypothesis.

Taking the R² values from the JDK releases (shown in Table 5) as a sample, we statistically calculated the 95 percent confidence interval for the mean R² value to be 25.18 to 38.87 percent. The interval calculated indicates an overall low accuracy in predicting dependant variables.

We conclude that the OO metrics cannot predict maintenance effort in the framework evolution process because a new release of the framework is usually a major rework in which the majority of changes account for enhancing the functionality rather than bug fixing. However, we acknowledge that framework systems that exhibit different evolution characteristics from JDK can yield different results.

Using the data we collected from the two Java systems, we ran the regression test on each story. Because we are measuring the line changes in each class, we only considered the classes that are available in two consecutive iterations, even with name changes. Only Story 5 and Story 6 in System 1 have enough data for MLR. Table 6 shows the R² value for the dependant variables.

XP-like processes have relatively independent and self-contained stories. However, story development effort can be predicted because each story is an addition to the current system release that is an accumulation of all other stories.

Taking the R² values for System 1 (shown in Table 6) as a sample, we statistically calculated the 95 percent confidence interval for the mean R² value to be 94.87 to 99.43 percent. The interval calculated indicates an overall high accuracy in predicting dependant variables.

Table 7 shows the R² values of MLR in Stories 4 and 5 and Stories 6 and 7 of System 2, respectively. A reasonable number of the dependent variables can be predicted with significance.

Taking the R² values for System 2 (shown in Table 7) as a sample, we statistically calculated the 95 percent confidence interval for the mean R² value to be 78.95 to 93.43 percent. The interval calculated indicates an overall fairly reasonable accuracy in predicting dependant variables.

TABLE 6
MLR Results for System 1

System 1	Acronyms	Sample size = 11
		Story 5-6
		R ² %
Lines Added	LA	95.6
Lines Changed	LC	97.9
Lines Deleted	LD	88.5

TABLE 7
MLR Results of the System 2

System 2		Acronyms	Sample size = 11	Sample size = 17
			Story 4-5	Story 6-7
			R ² %	R ² %
Lines Added	LA		97.7	54.2
Lines Changed	LC		98.4	72.9
Lines Deleted	LD		71	51.5

The MLR tests for Systems 1 and 2 accept Hypothesis 2 in a limited context: the prediction of the dependent variables is fairly significant only toward the last several iterations in the XP-like process. We conclude that in most cases, OO metrics can fairly predict maintenance efforts only toward the end of the iteration process when the system architecture has reasonable shape. The prediction is impossible at the beginning of the process because there is very little substantive design structure in the system.

Tables 8 and 9 present the test results for Hypotheses 3 and 4 with the R² values for Stories 5 and 6 in System 1 and Stories 4, 5, 6, and 7 in System 2. R² values for most of the stories are fairly reasonable. The values for Story 6 in System 2 are unusually low; this may be explained by noting the very big spike in the number of classes (sample size increased by 33 classes) in a very short development duration of only five days. This jump indicates instability in the system in that duration [25]. In most stories, the refactoring effort can be predicted with reasonable accuracy; in most stories, the error fix effort can be predicted with reasonable accuracy. We conclude that the OO metrics can fairly predict the refactoring and error-fix effort in the XP-like process, but only when the system designs accumulate to certain mass.

5 CONCLUSION

We presented an empirical study on the effectiveness of OO metrics in two iterative processes. We found out that OO metrics are reasonably effective in predicting evolution

changes of a system design in the short-cycled XP-like process, but they are largely ineffective in the long-cycled framework evolution process. Furthermore, the predictive capability of OO metrics in the short-cycled iterative process is only effective when the design accumulates to a certain critical mass, often toward the last several iterations in the XP-like process. This study provides further evidence on the abilities and limitations of OO metrics in predicting maintenance effort.

ACKNOWLEDGMENTS

The authors would like to thank the anonymous reviewers for their constructive contributions. This work is supported in part by a grant from the US National Science Foundation (NSF CCR-0097058).

REFERENCES

- [1] S.W. Ambler, *Agile Modeling: Effective Practices for Extreme Programming and the Unified Process*. Wiley, 2002.
- [2] V. Basili, L. Briand, and W.L. Melo, "A Validation of Object-Oriented Design Metrics as Quality Indicators," *IEEE Trans. Software Eng.*, vol. 22, no. 10, pp. 267-271, Oct. 1996.
- [3] V. Basili, L. Briand, S. Condon, Y. Kim, W. Melo, and J. Valett, "Understanding and Predicting the Process of Software Maintenance Releases," *Proc. Eighth Int'l Conf. Software Eng. (ICSE-18)*, Mar. 1996.
- [4] K. Beck, "Embracing Changes with Extreme Programming," *Computer*, pp. 70-77, Oct. 1999.
- [5] S. Weisberg, *Applied Linear Regression*. John Wiley & Sons, 1985.
- [6] W. Li and S. Henry, "Maintenance Metrics for Object-Oriented Paradigm," *Proc. First Int'l Software Metrics Symp.*, pp. 52-60, May 1993.
- [7] F. Fioravanto and P. Nesi, "Estimation and Prediction Metrics for Adaptive Maintenance Effort of Object-Oriented Systems," *IEEE Trans. Software Eng.*, vol. 27, no. 12, pp. 1062-1084, Dec. 2001.
- [8] L. Briand, S. Morasca, and V. Basili, "Defining and Validating Measures for Object-Based High-Level Design," *IEEE Trans. Software Eng.*, vol. 25, no. 6, pp. 722-743, June 1999.
- [9] P. Nesi and T. Querci, "Effort Estimation and Prediction of Object-Oriented Systems," *J. Systems and Software*, vol. 42, no. 1, pp. 89-102, 1998.
- [10] S.R. Chidamber and C.F. Kemerer, "A Metrics Suite for Object-Oriented Design," *IEEE Trans. Software Eng.*, vol. 20, no. 6, pp. 476-493, June 1994.
- [11] S. Moser, B. Henderson-Sellers, and V. Misić, "Cost Estimation Based on Business Models," *J. Systems and Software*, vol. 49, no. 1, pp. 33-42, 1999.
- [12] T. Hastings and A. Sajeew, "A Vector-Based Approach to Software Size Measurement and Effort Estimation," *IEEE Trans. Software Eng.*, vol. 27, no. 4, pp. 337-350, Apr. 2001.
- [13] L. Briand and J. Wüst, "Modeling Development Effort in Object-Oriented Systems Using Design Properties," *IEEE Trans. Software Eng.*, vol. 27, no. 11, pp. 963-986, Nov. 2001.
- [14] H. Rombach, "A Controlled Experiment on the Impact of Software Structure On Maintainability," *IEEE Trans. Software Eng.*, vol. 13, no. 3, pp. 334-354, Mar. 1987.

TABLE 8
XP Activities Effort Prediction—System 1

System 1			Refactoring	Error Fix
	Sample Size	Development Duration in days	R ² %	R ² %
Story 5	19	94	76.5	88.3
Story 6	30	123	72.9	69.4

TABLE 9
XP Activities Effort Prediction—System 2

System 2			Refactoring	Error Fix
	Sample Size	Development Duration in days	R ² %	R ² %
Story 4	17	12	61	44.5
Story 5	19	24	66.0	74.4
Story 6	52	5	14.3	10.0
Story 7	50	30	91.2	90

- [15] J. Fox, *Applied Regression Analysis, Line Models, and Related Methods*. AGE Publication, 1997.
- [16] H. Rombach, "Design Measurement, Some Lessons Learned," *IEEE Software*, vol. 7, no. 2, pp. 17-25, Mar./Apr. 1990.
- [17] M. Hitz and B. Montazeri, "Chidamber and Kemerer's Metrics Suite: A Measurement Theory Perspective," *IEEE Trans. Software Eng.*, vol. 22, no. 4, pp. 267-271, Apr. 1996.
- [18] W. Li and S. Henry, "Object Oriented Metrics that Predict Maintainability," *J. Systems and Software*, vol. 23, no. 2, pp. 111-122, Nov. 1993.
- [19] W. Li, "Another Metric Suite for Object-Oriented Programming," *The J. Systems and Software*, pp. 155-162, 1998.
- [20] N. Fenton and M. Neil, "A Critique of Software Defect Prediction Models," *IEEE Trans. Software Eng.*, vol. 25, no. 5, pp. 675-689, Sept./Oct. 1999.
- [21] D.C. Montgomery and E.A. Peck, *Introduction to Linear Regression Analysis*, second ed. Wiley series in probability and mathematical statistics, 1992.
- [22] N. Fenton and N. Ohlsson, "Quantitative Analysis of Faults and Failures in a Complex Software System," *IEEE Trans. Software Eng.*, vol. 26, no. 8, pp. 797-814, Aug. 2000.
- [23] N.I. Churcher and M.J. Shepperd, "Comments on a Metrics Suite for Object Oriented Design," *IEEE Trans. Software Eng.*, vol. 21, no. 3, pp. 263-265, Mar. 1995.
- [24] K. El Emam, W. Melo, and J. Machado, "The Prediction of Faulty Classes Using Object-Oriented Design Metrics," *J. Systems and Software*, vol. 56, no. 1, pp. 63-75, 2001.
- [25] W. Li, L. Etzkorn, C. Davis, and J. Talburt, "An Empirical Study of Object-Oriented System Evolution," *Information and Software Technology*, vol. 42, no. 6, pp. 373-381, Apr. 2000.
- [26] R. Harrison, S. Counsell, and R. Nithi, "An Investigation into the Applicability and Validity of Object-Oriented Design Metrics," *Empirical Software Eng.*, vol. 3, no. 3, pp. 255-273, 1998.
- [27] M. Alshayeb, W. Li, and S. Graves, "An Empirical Study of Refactoring, New Design, and Error-Fix Efforts in Extreme Programming," *Proc. Fifth World Multiconf. Systemics, Cybernetics and Informatics (SCI 2001)*, 2001.
- [28] W. Li and J. Talburt, "Empirically Analysing Object-Oriented Software Evolution," *J. Object-Oriented Programming (JOOP)*, pp. 15-19, Sept. 1998.



member of the IEEE and the IEEE Computer Society.

Mohammad Alshayeb received the MS and PhD degrees in computer science and the certificate of software engineering from the University of Alabama in Huntsville, and the BS degree in computer science from Mutah University, Jordan. He is a senior research associate at the Information Technology and Systems Center, University of Alabama. His research interests include software metrics, software quality, and object-oriented design. He is a



Wei Li received the BS degree in computer science from Beijing University and the PhD degree from Virginia Polytechnic Institute and State University, Blacksburg. He is an associate professor in the Computer Science Department at University of Alabama in Huntsville. Dr. Li is a research fellow at the Institute of Software and a professor at the Graduate School, Chinese Academy of Sciences, Beijing. He is a member of the IEEE and the IEEE Computer Society.

► For more information on this or any computing topic, please visit our Digital Library at <http://computer.org/publications/dlib>.